THE COMPLETE HANDS-ON GUIDE

PYTHON MASTERY

From Zero to Pro

BCA STUDENTS | BEGINNER FRIENDLY | 100+ TESTED CODES | PROJECTS + Q&A



WRITTEN BY

Abhishek Prajapati

BCA Student • Entrepreneur • Content Creator

Published by NexCart Publications | First Edition 2026

How To Use This Book

Every chapter follows the same simple pattern: **Concept → Diagram → Tested Code → Output → Practice**. Here is what the coloured elements mean:

PRO TIP

Shortcuts and good habits that professionals use.

NOTE

Extra information that helps you understand the topic better.

WARNING

Common mistakes and traps. Read these carefully!

REMEMBER

Key points to revise before exams and interviews.

INTERVIEW / EXAM

Questions that are often asked in viva, exams and interviews.

Code appears inside a **dark frame** with coloured dots. Directly below it, in a **green output box**, you see the real output printed when the code was run.

Your Learning Roadmap



Follow the chapters in order. Chapters 1 to 9 build your foundation, 10 to 13 make you a confident problem-solver, 14 to 15 take you to professional-level Python, and Chapters 16 and 17 give you six complete projects, interview questions, worked solutions and a cheat sheet.

Table of Contents

Chapter 1: Welcome to Python 9
- 1.1 What is Python? 9
- 1.2 Where is Python Used? 9
- 1.3 Installing Python 10
- 1.4 Where Should You Write Code? 10
- 1.5 Your First Program 10
- 1.6 How Python Runs Your Code 11
- 1.7 Comments, Indentation and Errors 11
- 1.8 Python Keywords 12

Chapter 2: Variables & Data Types 14
- 2.1 What is a Variable? 14
- 2.2 Rules for Naming Variables 14
- 2.3 Basic Data Types 15
- 2.4 Multiple Assignment and Swapping 16
- 2.5 Type Conversion (Casting) 16
- 2.6 Mutable vs Immutable (Quick Idea) 17
- 2.7 Constants 18

Chapter 3: Operators 19
- 3.1 Arithmetic Operators 19
- 3.2 Comparison Operators 20
- 3.3 Logical Operators 20
- 3.4 Assignment Operators 21
- 3.5 Membership and Identity Operators 22
- 3.6 Bitwise Operators (Overview) 22
- 3.7 Operator Precedence 23
- 3.8 Mini Example: Simple Interest 24

Chapter 4: Input, Output & Formatting 25
- 4.1 The print() Function in Depth 25
- 4.2 Taking Input from the User 26
- 4.3 f-Strings: The Modern Way to Format 27
- 4.4 Other Formatting Methods 29
- 4.5 Making a Neat Table 29
- 4.6 Mini Program: Shopping Bill 29

Chapter 5: Strings 31
- 5.1 Creating Strings 31
- 5.2 Indexing and Slicing 31
- 5.3 Strings are Immutable 33
- 5.4 Most Useful String Methods 33
- 5.5 Combining and Repeating Strings 34
- 5.6 Looping Through a String 35
- 5.7 Classic Examples 35

Chapter 6: Conditional Statements 37
- 6.1 The if Statement 37
- 6.2 if ... else 37
- 6.3 if ... elif ... else 38
- 6.4 Nested if 39
- 6.5 Ternary (One-line) if 39
- 6.6 Truthy and Falsy Values 39

6.7 match-case (Python 3.10+) 40
6.8 Practical Examples 40

Chapter 7: Loops 43
7.1 The for Loop 43
7.2 The while Loop 44
7.3 break, continue and pass 45
7.4 else with Loops 45
7.5 Nested Loops 46
7.6 Popular Loop Programs 47

Chapter 8: Lists & Tuples 50
8.1 Creating and Accessing Lists 50
8.2 Changing a List 51
8.3 Important List Methods 51
8.4 Looping Through Lists 52
8.5 Copying Lists Correctly 52
8.6 2D Lists (Matrices) 53
8.7 Tuples 53
8.8 Packing, Unpacking and Slicing Tricks 54
8.9 Mini Program: Student Marks Analyser 55

Chapter 9: Dictionaries & Sets 56
9.1 Creating a Dictionary 56
9.2 Adding, Updating and Deleting 57
9.3 Looping Through a Dictionary 57
9.4 Useful Dictionary Methods 58
9.5 Nested Dictionaries 58
9.6 Dictionary Comprehension and Counting 59
9.7 Sets 59
9.8 Which Collection Should I Use? 60
9.9 Mini Program: Phone Book 61

Chapter 10: Functions 62
10.1 Defining and Calling a Function 62
10.2 Parameters and Return Values 62
10.3 Default and Keyword Arguments 63
10.4 *args and **kwargs 63
10.5 Returning Multiple Values 64
10.6 Variable Scope 64
10.7 Docstrings and Type Hints 65
10.8 Recursion 65
10.9 Common Pitfall: Mutable Default Argument 66
10.10 Useful Function Examples 66

Chapter 11: Comprehensions, Lambda & Built-ins 68
11.1 List Comprehension 68
11.2 Set and Dictionary Comprehensions 69
11.3 Lambda (Anonymous) Functions 69
11.4 map(), filter() and reduce() 70
11.5 zip(), enumerate(), any(), all() 70
11.6 Useful Built-in Functions 71
11.7 Generators and yield 71
11.8 Mini Program: Data Cleaning 72

Chapter 12: File Handling 74
12.1 Opening Files and Modes 74
12.2 Writing and Reading a Text File 74

12.3 Appending to a File 76
12.4 Checking Files with os and pathlib 76
12.5 Working with CSV Files 76
12.6 Working with JSON Files 77
12.7 Mini Program: Daily Expense Tracker 78

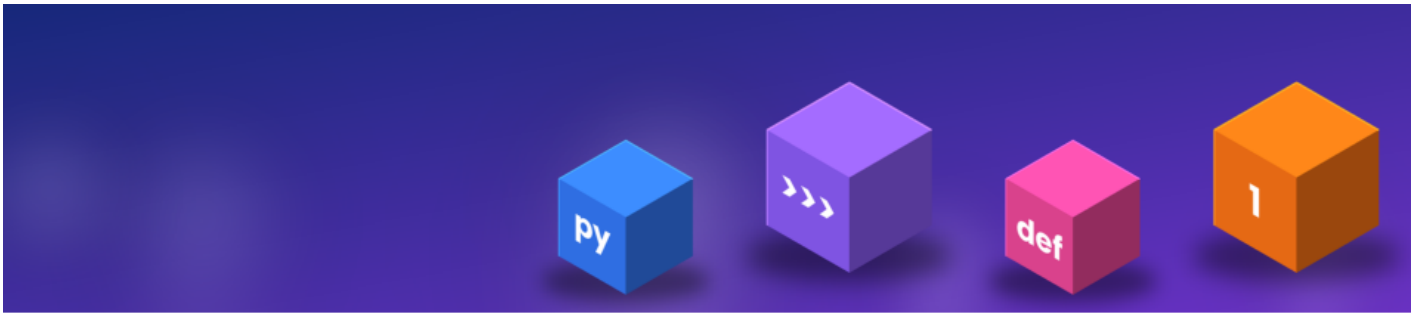
Chapter 13: Error & Exception Handling 79
13.1 Types of Errors 79
13.2 Common Exceptions 79
13.3 try and except 80
13.4 else and finally 81
13.5 Catching Many Errors at Once 82
13.6 Raising Your Own Exceptions 82
13.7 Input Validation Loop 83
13.8 assert and Debugging Tips 84

Chapter 14: Object-Oriented Programming 85
14.1 Classes and Objects 85
14.2 Instance vs Class Attributes 86
14.3 The __str__ Method 87
14.4 Encapsulation 87
14.5 Properties (Getter / Setter the Pythonic Way) 88
14.6 Inheritance 88
14.7 Class Methods and Static Methods 90
14.8 Magic (Dunder) Methods 90
14.9 Dataclasses (Modern Shortcut) 91
14.10 Mini Program: Library System 91

Chapter 15: Modules, Libraries & pip 93
15.1 What is a Module? 93
15.2 Importing Modules 93
15.3 Useful Standard-Library Modules 94
15.4 Creating Your Own Module 96
15.5 pip: The Package Installer 97
15.6 Virtual Environments 97
15.7 Popular Third-Party Libraries 98

Chapter 16: Six Complete Mini Projects 99
16.1 Project 1: Smart Calculator 99
16.2 Project 2: Number Guessing Game 100
16.3 Project 3: To-Do List Manager (with File Saving) 101
16.4 Project 4: ATM / Bank System (OOP) 102
16.5 Project 5: Quiz Application 104
16.6 Project 6: Password Generator & Strength Checker 105
16.7 Ideas to Extend Your Projects 105

Chapter 17: Interview Prep, Solutions & Cheat Sheet 107
17.1 Top Python Interview Questions 107
17.2 Solutions to Selected Practice Questions 110
17.3 Python Syntax Cheat Sheet 114
17.4 Common Errors and Their Fixes 114
17.5 Your 30-Day Practice Plan 115
17.6 What to Learn Next 115



CHAPTER 01

Welcome to Python

> Your first step into the world of programming

Python is one of the easiest and most powerful programming languages in the world. In this chapter you will understand what Python is, install it, and run your very first program.

1.1 What is Python?

Python is a high-level, general-purpose programming language created by Guido van Rossum and first released in 1991. It is famous for its clean, English-like syntax, which makes it the best first language for beginners and a favourite of professionals.

Feature	What it means for you
Easy to read	Code looks like simple English, so you learn faster.
Interpreted	Python runs your code line by line. No separate compile step.
Dynamically typed	You do not have to declare variable types.
Huge libraries	Ready-made tools for web, data, AI, games and automation.
Cross-platform	Same code runs on Windows, macOS and Linux.
Free & open source	Download and use it without paying anything.

1.2 Where is Python Used?

- **Web development** — Django, Flask, FastAPI
- **Data science & analytics** — Pandas, NumPy, Matplotlib
- **Artificial Intelligence & Machine Learning** — scikit-learn, TensorFlow, PyTorch
- **Automation & scripting** — renaming files, sending emails, web scraping
- **Games, apps and cyber-security tools**

PRO TIP

As a BCA student, Python is useful in almost every semester: programming practicals, data structures, DBMS projects, AI/ML subjects and your final-year project.

1.3 Installing Python

1. Open the official website **python.org** and go to **Downloads**.
2. Download the latest **Python 3** installer for your system.
3. Windows users: on the first screen, **tick "Add python.exe to PATH"**, then click *Install Now*.
4. Open Command Prompt (Windows) or Terminal (macOS/Linux) and type the version command below.

terminal

```
python --version
# On macOS / Linux you may need:
python3 --version
```

WARNING

If your computer says `python is not recognized`, you forgot to tick "Add to PATH". Run the installer again and choose *Modify* / *Repair*.

1.4 Where Should You Write Code?

Tool	Best for	Cost
IDLE	Comes with Python; good for day one	Free
VS Code	Most popular editor; great extensions	Free
PyCharm Community	Full-featured Python IDE	Free
Google Colab	Run Python in the browser, no install	Free
Jupyter Notebook	Data science and experiments	Free

1.5 Your First Program

Tradition says that the first program in any language prints "Hello, World!". In Python it takes just one line. `print()` is a built-in **function** that displays text on the screen.

hello.py — Python 3

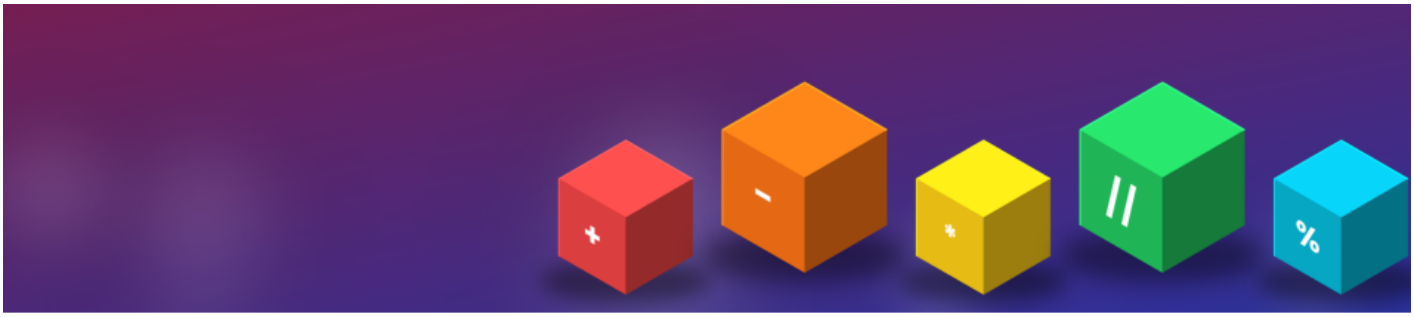
```
print("Hello, World!")
```

OUTPUT

```
Hello, World!
```

PRACTICE ZONE

1. Print your name, your city and your course on three separate lines.
2. Print a small box made of `*` characters using several print statements.
3. Add a comment above each line of your program explaining what it does.
4. Deliberately remove a quote from a print statement and read the error message.
5. Check which Python version is installed on your system.



CHAPTER 03

Operators

Do maths, compare values and build conditions

Operators are special symbols that perform operations on values. You will use them in every single program you write.

3.1 Arithmetic Operators

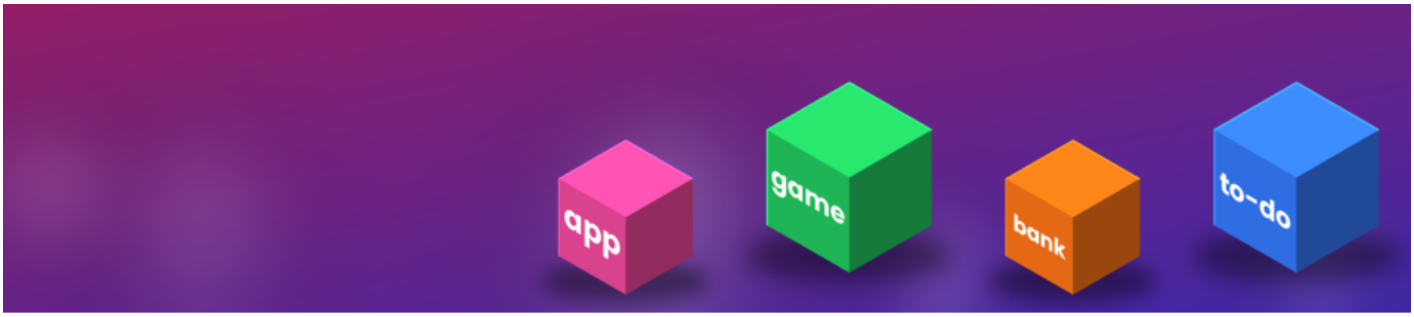
Operator	Name	Example	Result
+	Addition	10 + 3	13
-	Subtraction	10 - 3	7
*	Multiplication	10 * 3	30
/	Division (float)	10 / 3	3.333...
//	Floor division	10 // 3	3
%	Modulus (remainder)	10 % 3	1
**	Power	10 ** 3	1000

arithmetic.py — Python 3

```
a = 17
b = 5
print("a + b =", a + b)
print("a - b =", a - b)
print("a * b =", a * b)
print("a / b =", a / b)
print("a // b =", a // b)
print("a % b =", a % b)
print("a ** b =", a ** b)
print(divmod(a, b))
```

OUTPUT

```
a + b = 22
a - b = 12
a * b = 85
a / b = 3.4
a // b = 3
```



CHAPTER 16

Six Complete Mini Projects

Put everything together and build real programs

Reading is not enough — builders learn by building. Here are six complete projects. Each one is tested, explained and ready to run. Type them yourself, then add your own features to make them unique. They also look great in your resume and college project file.

16.1 Project 1: Smart Calculator

Concepts used: functions, dictionary, loops, exception handling, f-strings.

PROGRAM FLOW

STEP 1	STEP 2	STEP 3	STEP 4	STEP 5
Show menu + - * / % **	Take input two numbers	Calculate via function	Show result or error	Repeat until quit

calculator.py — Python 3

```
def add(a, b): return a + b
def sub(a, b): return a - b
def mul(a, b): return a * b
def div(a, b):
    if b == 0:
        raise ZeroDivisionError("Cannot divide by zero")
    return a / b

operations = {"+": add, "-": sub, "*": mul, "/": div}

print("=== SMART CALCULATOR ===")
while True:
    op = input("Operator (+ - * /) or q to quit: ").strip()
    if op.lower() == "q":
        print("Goodbye!")
        break
    if op not in operations:
        print("Invalid operator")
        continue
    try:
        a = float(input("First number : "))
        b = float(input("Second number: "))
        print(f"Result: {a} {op} {b} = {operations[op](a, b):g}")
    except ValueError:
```

17.5 Your 30-Day Practice Plan

Days	Focus	Daily goal
1 – 5	Chapters 1–4: Basics, variables, operators, I/O	Type every example, solve all practice
6 – 10	Chapters 5–7: Strings, if/else, loops	5 pattern and 5 string programs
11 – 15	Chapters 8–9: Lists, tuples, dicts, sets	Mini programs with data
16 – 20	Chapters 10–12: Functions, comprehensions, files	Rewrite old programs using functions
21 – 25	Chapters 13–15: Errors, OOP, modules	Build one small class-based program
26 – 30	Chapters 16–17: Projects and interview prep	Finish 2 projects, upload to GitHub

17.6 What to Learn Next

- **Data Structures & Algorithms** in Python for placements and coding interviews.
- **SQL and SQLite** with Python for database projects.
- **Web development** with Flask or Django.
- **Data analysis** with NumPy, Pandas and Matplotlib.
- **Automation**: Excel, PDF, email and web scraping scripts.
- **Git and GitHub** to build a public portfolio.

REMEMBER

Consistency beats intensity. One hour of coding every day is far better than ten hours once a week. Build projects, make mistakes, fix them — that is exactly how professionals are made.

THANK YOU

Thank you for choosing this book. I hope it helps you write your first program, ace your exams and build projects you are proud of. Keep learning, keep coding!
— Abhishek Prajapati